

composants matériels du processeur

CPU vs GPU: sequential versus parallel computing

↳ designed for tasks that requires parallel computing (multithreaded code)

↳ designed ~~for~~ to process instructions sequentially.

La RAM contient le noyau du système d'exploitation (kernel), qui est un logiciel chargé en RAM pour gérer tous ces composants matériels

Le reste de l'OS est utilisé par l'utilisateur mais c'est le noyau qui fait tout fonctionner.

Il existe deux types de processus:

processus systèmes ("superutilisateurs")

aussi appelés processus démons  
(puisqu'ils tournent indéfiniment dans le système pour servir les requêtes utilisateurs)

processus utilisateurs

processus lancés par les programmes utilisateurs

\* Daemon :

Disk and Extension Monitor (se termine souvent par "d")

L'ensemble des processus du système est représenté en général par une liste chaînée dont chaque élément est un bloc de contrôle ~~de~~ de processus (PCB) stocké dans la mémoire ~~noyau~~ est représentant un processus (contrôle d'accès)

|                                         |
|-----------------------------------------|
| Process-id                              |
| état                                    |
| priorité                                |
| droits                                  |
| pointeur vers adresse mémoire virtuelle |
| événement attendus                      |
| pointeur vers prochain                  |
| suivant <del>est</del>                  |
| code retour                             |

Segments qui vont être chargés en pages et stockés en RAM

organisation mémoire des processus:

|                                            |
|--------------------------------------------|
| U-Area                                     |
| Stack                                      |
| Point de rupture                           |
| Heap                                       |
| cbss (uninitialized)<br>data (initialized) |
| Code/text                                  |

↳ contient l'appel système en cours, les descripteurs de fichiers, le repertoire courant, la table limite, la sauvegarde des processus terminés etc...

↳ peut être déplacé suivant si le bas est entièrement occupé

↳ code exécutable du processus

C'est un système d'abstraction pour bien faire la distinction entre les processus par le noyau (le empêcher à un processus d'accéder à la mémoire d'un autre)

fin de contrôler la bonne exécution d'un processus, on impose à celui-ci de retourner au processus qui l'a lancé (processus père), un code retour (lorsqu'il se termine par convention). C'est une fin normale, tout autre état relatif signal un comportement anormal rencontré en cours d'exécution

La mémoire virtuelle est découpée en pages (souvent 4k o), la mémoire RAM est découpée en ~~en~~ cadre (de même taille). Les adresses virtuelles sont mappées sur des adresses physiques par le MMU (memory unit managing) quand elles seront ~~des~~ référencées par le processus.

- Le CPU génère une adresse virtuelle
- Le composant matériel MMU traduit cette adresse virtuelle en <sup>RAM</sup> adresse physique via la table des pages (stockée en RAM accessible <sup>seulement</sup> par le noyau)
- La RAM contient ces données et les exécute ou la page référencée n'est pas encore chargée en RAM ce qui produit un défaut de page où le noyau intervient pour y remédier :
  - l'os choisit une page à enlever de la mémoire afin d'en charger une autre, et si les données de la page à supprimer ont été modifiées, il fait les recopie sur le disk.
  - la page à supprimer est choisie via des algo de remplacement, sachant qu'il existe des processus d'os de pagination qui se réveille périodiquement pour inspecter l'état de la mémoire. Si trop peu de cases vides, il effectue un algo de remplacement

| espace libre d'adressage : stack                                                                                                        | heap                                                                                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| allocation de variable locales, ou appels de fonctions : objets créés automatiquement qui sont libérés dès que la fonction est terminée | objets créés dynamiquement par le processus au cours de son exécution, utile pour structure complexe, metadata (runtime <sup>data</sup> , app metadata...) |
|                                                                                                                                         | glacier than stack memory due to manual allocation and garbage collection needed                                                                           |

### Allocation dynamique de la mémoire :

- "malloc" demande l'allocation d'un certain nombre d'octets. la valeur retournée est un pointeur sur le début de la zone allouée : `void * malloc (size_t taille)`
- "calloc" demande l'allocation d'un certain nombre d'élément de tout d'octets, à chaque octet est initialisé à 0 : `void * calloc (size_t nbEl, size_t taille)`
- "realloc" permet de changer la taille allouée au pointeur ptr qui doit avoir été préalablement défini via malloc/calloc par ex : l'espace rajouté n'est pas initialisé et la valeur retournée peut être différente de ptr : `void * realloc (void * ptr, size_t taille)`

Bloc A = malloc(100)  
 => Bloc A  $\xrightarrow{\text{lib}}$  Bloc libre  
 (avec un  $\phi$  sous Bloc libre)

Il faut noter que <sup>dans</sup> chaque bloc alloué, pour le chargement des blocs, le système utilise ~~plusieurs~~ plusieurs objets afin de pouvoir recourir aux algo - "free" libère l'espace d'adresse ptr  $\rightarrow$  void free(void \*ptr) ordonneurs de mémoire.  
 Pour libérer une zone occupée, on la fusionne avec les zones adjacentes déjà libre. Pour sélectionner une zone à allouer, on cherche souvent une zone de taille ~~plus~~ supérieure car il est rare d'avoir exactement le bon espace. Si aucune zone n'est convenable mais que la somme des zones libre est bien supérieure à la zone demandée, un algo de compactage (garbage collector) s'impose (regroupe les zones libres en un seul bloc et fait des déplacements de mémoire)

Un processus est défini comme un espace d'adressage et une unité d'exécution. Mais en le voyant à son orga mémoire, il devient clair d'en créer surtout dans le cadre de multiprocesseurs ou d'applications clients/serveurs où de nombreuses instances de processus sont créées. D'où la création de threads dont le contexte est réduit à l'essentiel (registres, priorité, signal, pile d'exécution, données et code d'exécution). D'ailleurs pour pouvoir partager des données, les processus partent par un segment de mémoire partagée (et un processus fils accède physiquement aux données de son père ~~sans avoir besoin de~~ <sup>et travailler une copie de</sup> celui-ci). Tandis que les threads issus d'un même processus peuvent partager des données communes déclarées au niveau du père sans avoir recours au segment partagé (par exemple le heap est partagé ~~par~~ <sup>avec</sup> tous les threads. Une thread occupe la mémoire de son processus mais est vue par le noyau comme une unité d'exécution distincte (processus légers TCB) et est donc ajoutée à la file d'attente du planificateur du noyau.

Des processus ou des threads n'appartenant pas au même processus peuvent bien sûr communiquer entre eux par tubes (pipes). Les pipes ~~sont~~ sont unidirectionnels, ainsi, les résultats d'un processus vont servir de données d'entrée pour le suivant. Pour établir une communication bidirectionnelle, il faut donc deux pipes. ~~Ces pipes sont créés dans~~ Mémoire tampon. Les pipes ne conservent pas les données une fois lues  $\rightarrow$  synchronisation essentielle des données reçues. Via la syntaxe suivante : commande<sub>1</sub> | commande<sub>2</sub> | commande<sub>3</sub> ... Les données sont écrites et lues aux ~~deux~~ extrémités du tube.

Pour éviter que certains processus accèdent aux mêmes ressources simultanément quand ils ne devraient pas, on utilise des sémaphores : il y a deux opérations atomique  $\uparrow$  (incrémente la valeur de la sémaphore) et  $\downarrow$  (décrémente). Tant que la valeur de la sémaphore est positive, les processus entrent, sinon ils sont mis en attente. D'où les sémaphores binaires si l'on veut qu'un seul processus ~~entree~~ accède aux ressources à un moment donné. (val = 0 ou 1)

Il y a aussi les mutex (mutual exclusion) qui sont des mécanismes de synchronisations et qui permet qu'un seul thread ou proces accède à une ressource partagée à la fois. Il agit comme un verrou (lock/unlock)

Ex: un certain nombre de processus/threads veulent accéder à une ressource, ils sont mis en attente par le mutex qui se lock une fois un processus arrive et ne peut être interrompu tant que le proces n'a pas fini (sorti du mutex) ensuite il ~~se~~ s'unlock et c'est celui d'un autre etc... (dans les jeux vidéos cela permet de ne pas décaler les objets quand 2 joueurs le prennent en même temps) (sert à ne pas perdre le même message aux deux si ce n'est pas le bon ou à ne pas écraser des données).

Sémaphore : sync de processus | mutex : exclusion mutuelle de thread  
gère par ~~un~~ thread

Pour la sync de processus il y a aussi les barrières : les processus qui terminent sont mis en attente jusqu'à ce que le dernier ait terminé avant de passer aux tâches suivantes, pratique quand les procesus sont dépendants.

Un système doit attribuer à chaque processus un temps procesus équitable, faire en sorte que tous les parties soient occupées (équilibre) et que les politiques définies soient appliquées. Pour cela il y a des algo d'ordonnancement : optimiser le nombre de jobs à l'heure, délai de rotation... Ex: FIFO (premier arrivé - premier servi), Earliest deadline first, exécution du job le plus court, exécution du job dont le temps restant est le plus court...

- Les systèmes interactifs ont besoin de répondre rapidement aux requêtes et de répondre aux attentes des utilisateurs : ordonnancement par le roulet (round robin) (via intervalles de temps), ordonnancement par priorité, ordonnancement équitable (dépend du proprio du processus) (proportionnelle)
- Les systèmes temps réel : zero tolerance (hard real time) et ceux avec tolerance (soft real time), deux type d'événements : périodiques et imprévisibles (ou aperiodiques) ne peut être ordonné que si

$$\text{nb d'événements} \rightarrow m \quad \sum_{i=1}^m \frac{C_i}{P_i} \leq 1$$

$C_i$  : temps de service  
 $P_i$  : période

- Tout processus qui se termine passe dans un état où il n'utilise plus aucune ressource si ce n'est une entrée dans la table des processus. Un processus entre dans l'état zombi lorsqu'il se termine alors que son père n'a pas encore pris connaissance de sa terminaison. Si le père se termine de façon anormale sans avoir récupéré le code retour du processus enfant, celui-ci restera indéfiniment dans l'état zombi, jusqu'à une intervention admin kill ou l'arrêt/reboot du système.

Un processus peut être interrompu par un signal (ex: CTRL-C)

- Un processus qui tourne en arrière-plan peut passer en premier plan et vice versa.
- Un processus peut continuer après la déconnexion de l'utilisateur.
- Un processus appartient à un utilisateur (uid) qui lui appartient à un groupe d'utilisateurs (gid) et il possède un identifiant unique (pid) attribué par le système à sa création. (par les threads on parle de TID)

Bit set-uid : si le programme associé à un processus a son bit set-uid positionné, les droits du processus sont ceux du propriétaire. Cela permet de d'accorder momentanément des droits. Ex : le fichier passwd n'est accessible que par le superutilisateur mais il faut que l'utilisateur puisse pouvoir changer son mdp, par cela le fichier contient la commande à son bit set-uid positionné pour ne modifier que le mdp de l'utilisateur qui l'exécute.

- multi cœurs : plusieurs unités de calcul dans un processeur
- multiprocesseurs : plusieurs processeurs dans un système  
ex : 2 processeurs de 4 cœurs = 8 cœurs et 100 threads → répartition des 100 sur les 8 cœurs en essayant de garder les threads d'un même processus sur le même cœur et avec la possibilité de migrer une thread vers un autre cœur si celui-ci est moins chargé.

Tout usage de la cryptographie nécessite des clés mais pour les conserver (ex: mdp BIOS) de manière sûre → puce TPM (Trusted Platform Module) qui peut réaliser des opérations cryptographiques (ex BitLocker), vérifier des signatures numériques etc...

La concurrence c'est plusieurs tâches qui doivent être exécutées, on peut les exécuter une par une <sup>(ou alternativement...)</sup> ~~ou~~ single thread ou on peut les exécuter en même temps (parallélisme  $\rightarrow$  multithreading).

A thread pool is a group of spawned threads that are waiting and ready to handle a task. When a program receives a task <sup>then</sup> it will assign one of the threads in the pool to the task, and that thread will process/execute the task.

Spawning unlimited threads for unlimited requests can be vulnerable to DOS attacks (this could create a havoc by using all our server's resources). We can decide to have a fixed number of threads waiting in the pool. The pool will maintain a queue of incoming requests. Each of the threads in the pool will pop off a request from this queue, handle the request and then ask the queue for another request while waiting in the pool.